ENGINEERING REPORT

Vehicle Location Data Product

From a mock vehicle-location signal to queryable velocity and acceleration. Four modules, each shown here as a black box first, then opened one module at a time. Source state read on 27 Oct 2026.

Module 1 · Generator	Module 2 · ETL + Loader	Module 3 · Use case	Module 4 · BI
Built and in main.	Built and in main.	Built and in main.	Built and in main.
`change 608136a`	`merged as a0c4756`	`merged as e896227`	`merged as 95a9f6f`

What the project does

The product turns a vehicle-location signal into motion data that people can query. The signal describes the host vehicle as a 1 m cube of 1,000,000 points (100 per side, 1 cm apart), captured 16 times a second. No real recordings exist for the exercise, so the project builds its own source and everything after it.

Four modules do the work. Each one reads what the previous one wrote and passes on a documented set of files.

1 · Generator
Turns a short scenario (a start position and a list of move and stop segments) into a drive: a point cloud for every frame and a manifest with the hash of each file.

2 · ETL + Loader
Checks the hashes, tests every frame against six error codes (VL-001 to VL-006), keeps 9 key points per frame (8 corners and the centre), and writes them to a Parquet lake. Bad frames go to a quarantine table.

3 · Use case
Computes the velocity and acceleration of the vehicle from the centre point, and stores them as a motion table.

4 · BI
One Streamlit app with four tabs: generate a drive, ingest it, compute its motion, inspect its data quality, see the motion, and run SQL on the lake.

The implemented use case is ego-motion extraction: the velocity and acceleration of a vehicle that drives forwards, drives backwards, stops, and changes speed. Five more use cases are written up in `docs/USE_CASES.md` and not built.

Engineering concepts and assumptions

Tags: DECISION a choice we made and can defend, ASSUMPTION something we took as given, MEASURED a number from a real run.

HOW THE SYSTEM IS CUT

One-way dependencies. DECISION
A module uses only the shared core and the modules before it. Each can be tested alone.

Files are the contract. DECISION
Module 1 hands over a drive folder (manifest schema version 2). Module 2 hands over lake tables. Each handover is a documented Parquet schema, so separate sessions built the modules in parallel.

Python SDK and SQL, no web service. DECISION
VLClient reads lake tables as Arrow and runs SQL with DuckDB. A REST API is listed as coming soon.

DATA AND IDENTITY

Scenario hash names the drive. DECISION
SHA-256 of the canonical scenario, including cube size and density. The first 12 hex characters are the folder name. The manifest keeps all 64 and the hash of every file. Same scenario, same folder.

Re-ingest is safe to repeat. DECISION
The lake is Hive-style Parquet, partitioned by drive. A drive is replaced by writing a new folder and renaming it into place. That is two renames, not one atomic step, and the docs say so.

ASSUMPTIONS ABOUT THE SIGNAL

Straight line along x. ASSUMPTION
Signed speed, no turning, y and z constant. The shape of a cube shows no orientation, so yaw could not be read back from the points.

No sensor noise. ASSUMPTION
The only error is rounding the points to whole centimetres.

Fixed point order. ASSUMPTION
Module 2 picks corners and centre by index. The corner spacing also serves as a rigid-shape check (VL-004).

Full density. DECISION
1,000,000 points per frame is 12,000,000 bytes (computed). Frames are written in batches of about 50 with zstd. The 1,000-frame demo is 287,134,433 bytes with zstd and 4,024,944,648 with snappy (measured). A scenario may have at most 2,000 frames.

ALGORITHM AND VERIFICATION

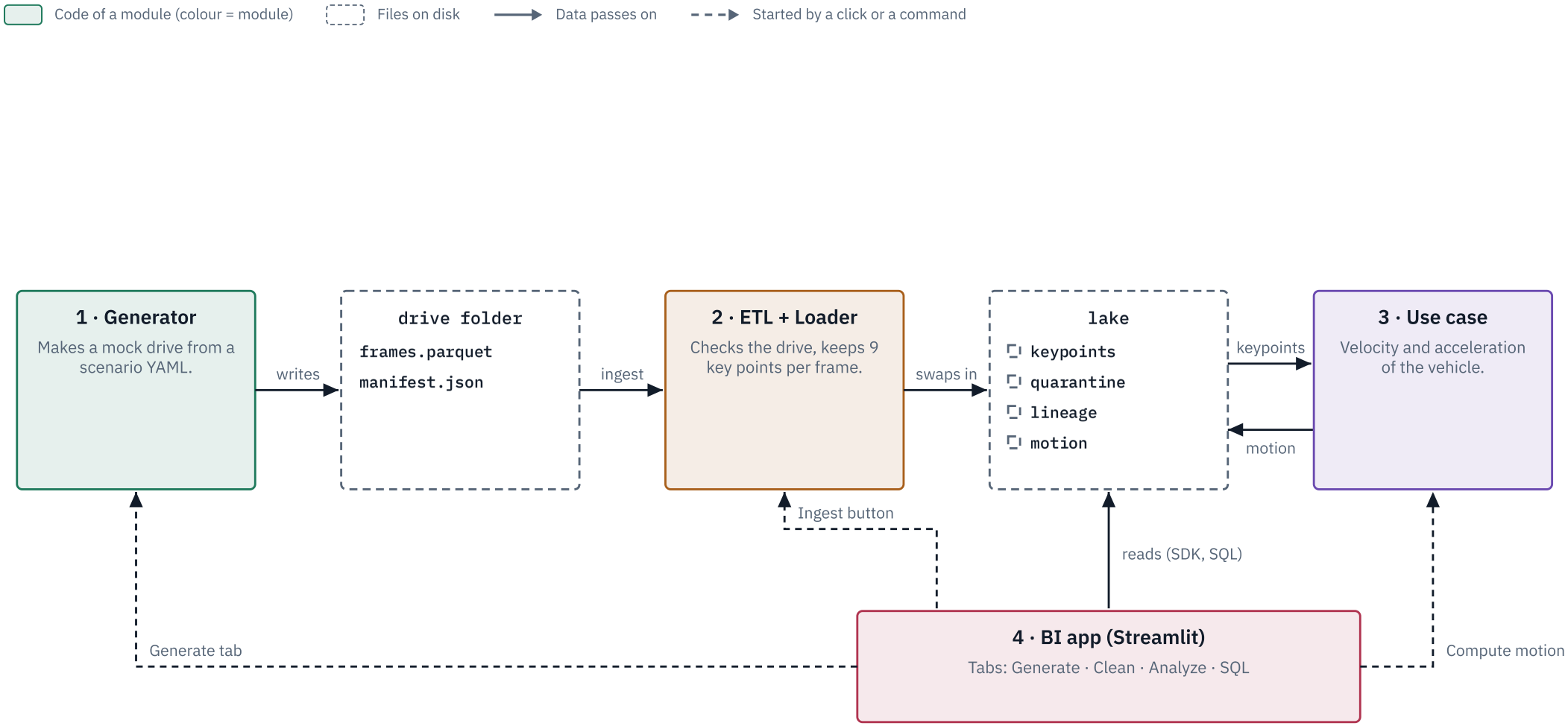
Window fit, not frame differences. COMPUTED
Positions are rounded to whole centimetres, so each can be off by 0.5 cm. A second difference at 16 fps multiplies that by 16 x 16 = 256, so two such errors can swing the acceleration by up to 512 cm/s², while the scenarios accelerate at about 100 cm/s². Velocity and acceleration therefore come from a sliding polynomial fit (16 frames, order 2).

The generator is tested by its inverse. DECISION
Property tests check that recovering a scenario from generated frames gives the scenario back, within measured tolerances.

Quality gates. MEASURED
pytest with hypothesis, ruff, mypy, Python 3.11. Module 1: 329 tests passed and 1 skipped on 26 Oct 2026; coverage was 100% at its last review gate, not re-run that day. Every assumption has a numbered entry in the project's assumptions log.

System flow. Read left to right: a scenario file becomes a drive folder, the drive folder becomes lake tables, and the lake tables become a motion table. The BI app reads the lake and has three buttons that start the other modules: Generate, Ingest and Compute motion. Orange squares in the lake are written by Module 2, violet by Module 3. The diagram is on the next page, turned sideways.

System flow


Code of a module (colour = module)
Files on disk
Data passes on
Started by a click or a command
1 · Generator
Makes a mock drive from a scenario YAML.
writes
drive folder
frames.parquet
manifest.json
ingest
2 · ETL + Loader
Checks the drive, keeps 9 key points per frame.
swaps in
lake
keypoints
quarantine
lineage
motion
keypoints
motion
3 · Use case
Velocity and acceleration of the vehicle.
Ingest button
reads (SDK, SQL)
Generate tab
4 · BI app (Streamlit)
Tabs: Generate · Clean · Analyze · SQL
Compute motion

File structure

Every tracked source, config, doc and tool file of the four modules, as in main on 27 Oct 2026.

Project root /

.streamlit/config.toml	Pins the Streamlit server to 127.0.0.1, because the SQL tab has no login.
.gitignore	Keeps the assignment PDF, generated data (data/, warehouse/, *.parquet), local environments, the LLM-usage logs and the local notes in docs/extras/ out of git.
README.md	Overview, quick start and a map of the documents.
pyproject.toml	Package definition (vl-data, Python 3.11+): dependencies and settings for pytest, ruff and mypy.

config/ config/

default.yaml	Shared defaults: 16 fps, raw data folder, cube of 100 cm with 100 points per axis, round-trip tolerances of the generator.
etl.yaml	Loader settings: accepted schema versions, read chunk size, size limits, lake folder (VL_LAKE_DIR overrides), top speed of 9,000 cm/s, keypoint list.
motion.yaml	Motion estimator settings: 16-frame window, polynomial order 2, minimum points and gap limits.

docs/ docs/

ARCHITECTURE.md	Flow, data model, technology choices with alternatives, storage and limits (about 2 pages).
ERROR_CODES.md	List of error codes VL-001 to VL-006 and INTEGRITY. Generated from the rule registry; a test fails if the file drifts.
LLM_USAGE.md	The model, the approach and the main prompts.
RELEASE_NOTES.md	Product description, usage with commands, known issues, coming soon.
USE_CASES.md	Six candidate use cases for the signal. One (ego-motion) is implemented.
extras/ENGINEERING_REPORT.pdf	This report as a PDF.

tools/ tools/

motion_sweep.py	Measures the motion estimator on generated drives to pick the window and order, and writes config/motion.yaml. Under review.

Package entry src/vl_data/

__init__.py	Package marker.
__main__.py	Entry point for python -m vl_data.
cli.py	Root command-line app (Typer). Mounts the commands of each module.

Shared core src/vl_data/core/

config.py	Configuration models and the YAML loader (cm, cm/s, frames). Checked on load.
hashing.py	Canonical JSON and SHA-256 helpers.
schemas.py	Arrow schemas of the drive files and the byte encoding of a point cloud.
manifest.py	Builds, writes, reads and verifies manifest.json (scenario hash plus the hash of every file).
lake.py	Hive-style Parquet lake: read, list, and replace the partition of one drive.
lake_schemas.py	Arrow schemas of the lake tables: keypoints, quarantine, lineage.

Module 1: generator src/vl_data/generator/

models.py	Scenario model: start state plus a list of move and stop segments. Strict validation.
body.py	The 1 m cube as a point lattice, and the indices of its corners.
motion.py	Position, speed and acceleration of every frame along x, from the segments.
drive.py	In-memory drive: the frames table.
writer.py	Writes a drive to disk in batches (zstd), checks it, then swaps the folder into place.
faults.py	Fault injection: drop frames, corrupt a frame, shift timestamps. Used to test the ETL.
inverse.py	Inverse of the generator: recovers track and segments from frames. Used by roundtrip.
cli.py	Commands generate, verify and roundtrip.
scenarios/demo.yaml	The 1,000-frame demo scenario (hash bf394b9bfb42).
ui/generate_tab.py	Generate tab: start form, segment table, speed preview, Generate button with size warning and disk-space check.
ui/generate_app.py	Standalone runner for the Generate tab (streamlit run).

Module 2: ETL and loader src/vl_data/etl/

config.py	Loader configuration models and loader for etl.yaml.
body.py	Reads the cube geometry from the manifest of a drive.
keypoints.py	Picks the 8 corners and the centre of the cube from a point cloud, by index.
facts.py	The single pass over the frames: one small fact row and the keypoints of each frame.
rules.py	Rule registry VL-001 to VL-006 and the runner that applies it.
ingest.py	Ingests one drive: verify hashes, validate, reduce to keypoints, swap the tables into the lake.
errors_doc.py	Builds docs/ERROR_CODES.md from the rule registry.

cli.py	Commands ingest and error-codes.

Module 3: read API and use case `src/vl_data/sdk/ and src/vl_data/usecase/`

sdk/client.py	VLClient: read-only access to the lake tables as Arrow tables, plus SQL (DuckDB).
usecase/config.py	Models of motion.yaml.
usecase/estimator.py	Sliding-window polynomial fit that gives velocity and acceleration from positions.
usecase/pipeline.py	Computes per-frame motion of the centre point and stores the motion table in the lake.
usecase/cli.py	Command motion.

Module 4: BI app `src/vl_data/bi/`

data.py	Data layer: drive list, quality numbers, commands and queries. No Streamlit.
figures.py	Plotly figures.
lake_lock.py	One in-process lock, shared by the Ingest and Compute motion buttons, so two writes to the lake never run at once in the app.
motion_panel.py	Motion panel of the Analyze tab: computes or recomputes the motion table of a drive on a click, through compute_motion.
ingest_panel.py	Ingest panel: lists raw drive folders and ingests one drive per click through ingest_drive. One of the app's two writes to the lake.
clean_tab.py	Clean tab: quality of one drive, kept and quarantined frames by error code. Also shows the Ingest panel.
analyze_tab.py	Analyze tab: the stored position, velocity and acceleration against time, and the Compute and Recompute motion buttons.
sql_tab.py	SQL tab: one read-only SELECT over the lake views, example queries, 10,000-row display cap.
shell.py	App shell: drive selector in the sidebar, four tabs, scenario-hash badge.
app.py	Runner: python -m streamlit run src/vl_data/bi/app.py.

Tests `tests/`

conftest.py, test_cli.py	Shared fixtures and tests of the root command-line app.
core/ (5 test files)	Config, hashing, schemas, manifest, body limits.
generator/ (9 test files)	Scenario model, body, motion, writer and full density, faults, inverse and round trip, CLI, Generate tab.
etl/ (9 test files)	Config, body, keypoints, facts, each rule, ingest, error-code document, CLI, end to end.
sdk/ and usecase/ test files	VLClient; config, estimator, pipeline, motion sweep, CLI, end to end.
bi/ (8 test files)	Data layer, figures, SQL tab, tabs, shell, ingest panel, motion panel, and a check that the app has no evaluation.

Not listed: generated data (`data/`, `warehouse/`), the assignment PDF and the local LLM-usage files. All are git-ignored on purpose.

Module 1 · Generator

Builds the mock source. Given a scenario, it produces a drive: the full point cloud of a moving 1 m cube for every frame, and a manifest that lets anyone check the files. It does not read any other module's data. The other modules treat it as a black box that writes one folder.

IN

A scenario YAML: a start state (x, y, z in cm, speed in cm/s) and a list of move and stop segments. Settings from `config/default.yaml`: 16 fps, cube of 100 cm, 100 points per axis.

OUT

A drive folder `data/raw/<hash12>/`. `frames.parquet`: one row per frame, 1,000,000 points in one binary column. `manifest.json`: the scenario, the cube, fps, counts, schema version 2, and the hash and size of each file.

USED BY

Module 2 reads the frames and the manifest, and uses the fault injector in its tests. Module 4 hosts the Generate tab.

Module 1 structure. The Generate tab and the generate command give a scenario to models.py. The frames flow through motion.py and drive.py to writer.py, which writes the drive folder. The command line has two more commands: verify checks the manifest, and roundtrip uses inverse.py to recover the scenario from the frames. faults.py damages a drive on purpose and is used by tests only. The diagram is in the appendix at the end of this report (Diagram 1, turned sideways).

How a drive is made

1. **Load and validate the scenario**
   A YAML file with a start state and a list of move and stop segments. A stop needs the vehicle at rest, durations must be whole frames, and a scenario has at most 2,000 frames.
2. **Hash it**
   SHA-256 of the canonical JSON of the scenario and the body (size and points per axis). The first 12 hex characters name the drive folder.
3. **Build the motion**
   A straight-line ramp of speed inside each segment, integrated into a position per frame.
4. **Build the frames**
   The 100 x 100 x 100 lattice, shifted by the rounded position, in batches of 50 frames so memory stays near one batch.
5. **Write and check**
   The frames go to a temporary folder, the manifest is written last, the folder is swapped into place, and the drive is verified.

Fault injection

The generator can damage a drive on purpose so that Module 2 can be tested against known errors. `inject_faults` returns a new drive and a count per error code. Only the frames table changes; the scenario hash is copied unchanged. A faulty drive still passes `verify_drive`, because its manifest is built from the faulty file.

FAULT	TRIGGERS	WHAT IT DOES TO THE FRAMES TABLE
DropFrame	VL-001	Deletes that frame's row. Other frame ids are not renumbered.
ShiftTimestamp	VL-002	Adds a shift to one timestamp. The shift must be smaller than one frame interval.
DuplicateFrame	VL-003	Inserts a copy of the row right after the original.
CorruptPoints	VL-004	Removes the last point (12 bytes) from one frame.
BreakSchema	VL-005	Renames the column points to pts.
Teleport	VL-006	Adds a distance to the x coordinate of every point of one frame.

Module 2 · ETL + Loader

Takes the drive folder Module 1 wrote and turns it into clean, queryable tables. It verifies every file, tests each frame against six error codes, keeps nine key points per frame, and replaces the drive's rows in a Parquet lake. Running it twice gives the same result.

IN

A drive folder `data/raw/<hash12>/` with `frames.parquet` and `manifest.json`. Settings from `config/etl.yaml`: the nine key points, the speed limit of 9,000 cm/s, the read chunk size.

OUT

Three lake tables, one folder per drive. `keypoints`: frame, timestamp, key point name, x, y, z. `quarantine`: rejected frames or problems, with an error code and a message. `lineage`: one row per drive with its status (ok or rejected), counts of frames expected, in, kept, rejected and missing, and the hash of the frames file.

USED BY

Module 3 reads the keypoints and lineage through VLClient. Module 4 reads quarantine and lineage for the Clean tab, queries all of them in the SQL tab, and starts the ingest from a button.

Module 2 structure. A drive folder goes through four steps: ingest.py checks the hashes, facts.py scans the frames once and picks the key points, rules.py applies the six error codes, and lake.py swaps the new tables in. etl.yaml gives the settings. body.py (the cube size) and errors_doc.py (writes ERROR_CODES.md) are not drawn. The diagram is in the appendix at the end of this report (Diagram 2, turned sideways).

How a drive is ingested

1. **Gates, cheapest first**
   The folder name must be a valid drive id. The manifest must read, its folder name must equal the first 12 hex characters of the scenario hash, the schema version must be supported, and the frame rate must divide a second. Only then is every file hash checked, the slow step. Any failure rejects the whole drive before a frame is read.
2. **Schema gate**
   Open frames.parquet, refuse any row group above the size cap, and compare the columns with the agreed frames schema (VL-005).
3. **One scan**
   Read the frames file once, in chunks. For each frame keep a small record: frame number, timestamp, payload size, payload hash, whether the points form the expected cube, and the centre. For each valid frame also pick the 9 key points, so the file is never read twice.
4. **Rules**
   Run VL-001 to VL-004 on the records. Run VL-006 only on frames that passed those, starting from a trusted frame so one teleported frame cannot poison the rest.
5. **Reduce**
   Keep the key-point rows of the accepted frames, sorted by frame number and then by the key-point list, and write them as one batch. The file bytes do not depend on the chunk size.
6. **Write**
   Write each table's new partition aside, then swap it in for the old one: keypoints, quarantine, then lineage. Lineage goes last and acts as the commit marker. A rejected drive drops its keypoints and gets a quarantine row and a lineage row with status rejected.

Module 3 · Use case

The use case: the velocity and acceleration of the vehicle. It reads the centre point of every frame from the lake, fits a sliding window on the real timestamps, and stores the result as a motion table. It also owns VLClient, the read-only way into the lake that Module 4 uses too.

IN

The lake tables `keypoints` and `lineage`, read through VLClient, and `config/motion.yaml`: a window of 16 frames, polynomial order 2, at least 9 valid points in a window, gaps up to 4 frames.

OUT

The `motion` table, one row per frame: frame, timestamp, position x, velocity, acceleration (null where the window is too sparse) and the number of points in the window. One partition per drive.

USED BY

Module 4 draws the table in the Analyze tab, queries it in the SQL tab, and starts the computation from a button. Anyone can query it with SQL through VLClient.

Module 3 structure. VLClient reads the centre-point rows of an ingested drive. pipeline.py checks the drive and hands the positions to estimator.py, which fits a sliding window of 16 frames on the real timestamps. The result is swapped into the lake as the motion table. motion.yaml sets the window. The diagram is in the appendix at the end of this report (Diagram 3, turned sideways).

How motion is computed

1. **Check the drive**
   The drive must have exactly one lineage row, and its status must be ok. Otherwise the command stops with a named error and writes nothing.
2. **Read the centre point**
   Through VLClient, take the rows of the centroid key point for the drive: one position per frame, with its real timestamp.
3. **Fit a window around each frame**
   Around every frame take a window of 16 frames on the real timestamps, so a missing frame is a gap and not a shifted sample. Fit a polynomial of order 2 by least squares. A window needs at least 9 valid points and no gap wider than 4 frames.

4 **Read off the motion**
The fitted curve gives the position, its slope is the velocity and its curvature is the acceleration. A frame whose window is too sparse gets a null velocity and acceleration.

5 **Store it**
Replace the drive's motion partition in the lake in one swap. A second run leaves the file byte for byte the same.

The motion table

COLUMN	TYPE	MEANING
`frame_id`	int64	Frame number.
`timestamp_ns`	int64	Time of the frame in nanoseconds.
`x_cm`	float64	Fitted position of the centre along x, in cm.
`velocity_cm_s`	float64, null allowed	Signed velocity in cm/s. Null where there is no estimate.
`acceleration_cm_s2`	float64, null allowed	Signed acceleration in cm/s². Null likewise.
`n_points`	int32	How many points the window held.

VLClient, the read-only SDK

The one way Modules 3 and 4 read the lake. A valid drive that is not in the lake gives an empty table; an invalid drive id raises an error.

METHOD	WHAT IT RETURNS
`drives()`	List the drives that have key points.
`keypoints(drive, keypoint=None)`	Key points of a drive, ordered by frame and key point name.
`motion(drive)`	The motion table of a drive.
`lineage(drive)`	The lineage row of a drive.
`quarantine(drive)`	The rejected frames and problems of a drive.
`sql(query)`	Run one SELECT over the table views with DuckDB. Closes its connection after every query.

Module 4 · BI

One Streamlit app that lets a person look at a drive: generate one, ingest it into the lake, compute its motion, see its data quality, see the motion, and ask the lake their own question in SQL. It reads the lake through VLClient. It writes to the lake in two places, each on a click and one drive at a time: the ingest panel (ingest_drive) and the motion panel (compute_motion).

IN
The lake through VLClient (key points, quarantine, lineage, motion, SQL), the raw drive folders for the Ingest panel, and a drive selector in the sidebar (lake drives only). The Generate tab is Module 1's screen.

OUT
Screens, and two writes, each on a click and one drive at a time: the Ingest panel ingests a raw drive through ingest_drive, and the Compute motion button writes the motion table through compute_motion (the same functions as the ingest and motion commands). Everything else only reads. The whole chain, Generate, Ingest, Compute motion, Analyze, runs from the app.

USED BY
People: a data scientist exploring one drive at a time.

Module 4 structure. The Generate tab writes a drive folder. Each tab reads the lake through VLClient. Two buttons write the lake on a click: Ingest (ingest_drive, Module 2) in the Clean tab and Compute motion (compute_motion, Module 3) in the Analyze tab. They share one lock, so two writes never overlap. The diagram is in the appendix at the end of this report (Diagram 4, turned sideways).

The four tabs

A sidebar selector picks the drive. Every tab shows the scenario hash of that drive as a badge. The app adds no error codes: the Clean tab shows VL-001 to VL-006 and INTEGRITY.

TAB	WHAT IT SHOWS	READS
Generate	A form for a scenario, a speed preview and a Generate button. Writes a drive folder to disk. This is Module 1's screen. Under it sits the Ingest panel (below).	Module 1 writer; raw folders
Clean	For the selected drive: frames expected, in, kept, rejected and missing, a bar chart of kept, rejected and missing frames, a bar chart of quarantine rows per error code, and the quarantine table (up to 200 rows, with the full count stated). The Ingest panel is also shown here, including on an empty lake.	lineage, quarantine
Analyze	The stored motion: position, velocity and acceleration against time. A drive ingested without motion shows "Motion is not computed yet" and the button Compute motion (the motion command stays visible as the command-line way). With motion it shows the three charts and the button Recompute motion. A drive that is not ingested, or was rejected, shows the reason and the ingest command, with no button. Old motion with a lineage that is not ok shows the charts and a warning, with no button.	motion; writes the lake
Ingest panel	Lists the raw drive folders: pending ones first (newest folder first), already ingested ones last, labelled re-ingest. The button "Ingest this drive" ingests one drive per click, inside a status block, through ingest_drive, the same function as the ingest command. A rejected drive is shown as a result (a warning), not as an error. Messages are fixed texts without file paths. It is hidden if the lake listing fails.	raw folders; writes the lake
SQL	One read-only SELECT over the views keypoints, quarantine, lineage and motion, with five example queries in a dropdown. The result shows up to 10,000 rows and states the total. A non-SELECT, a second statement or a bad query shows a one-line message.	all four tables

How it is checked

From the Module 4 documents.

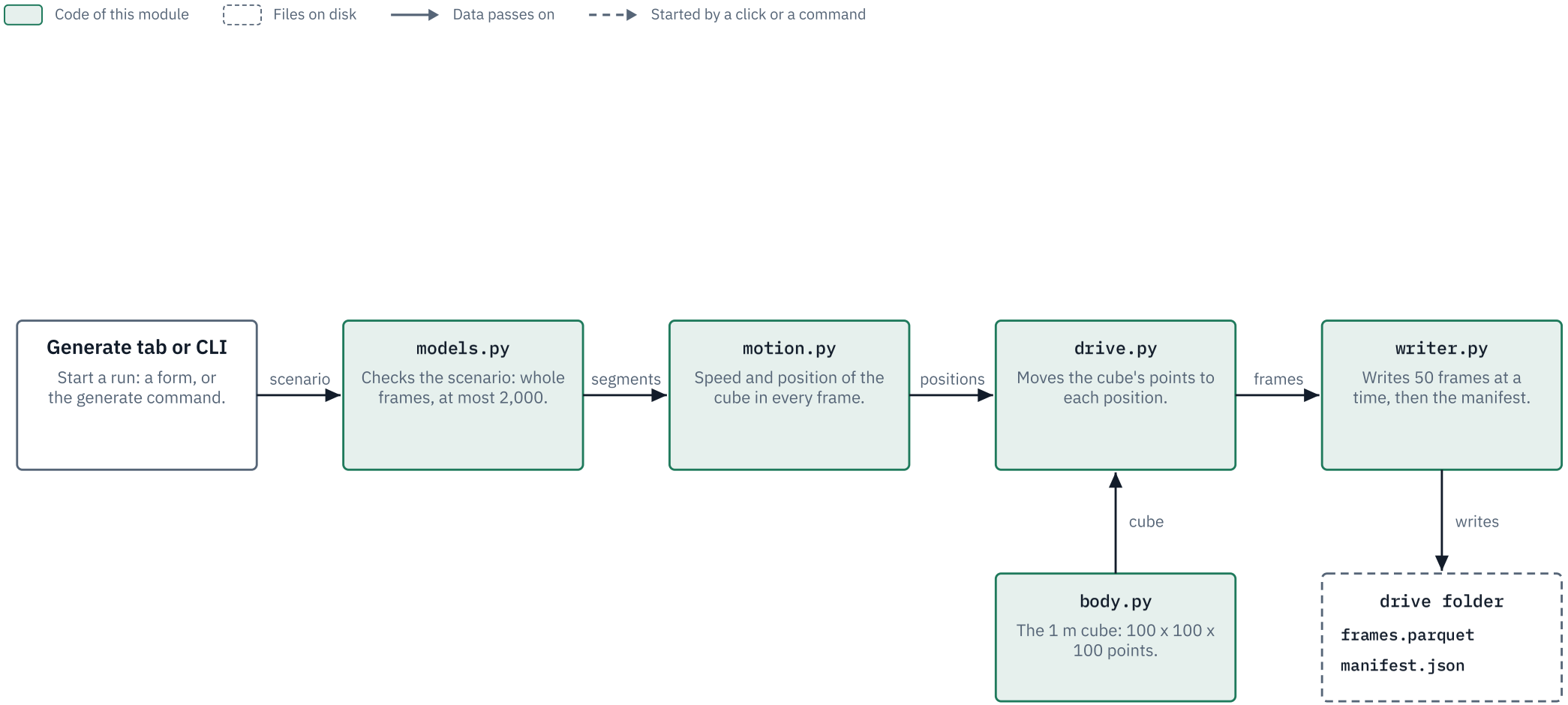
The tabs are tested with Streamlit's test runner on a small drive (a 100 cm cube of 10 points per axis). A live server was started once by a script: its health check, the page and the run of `app.py` answered, and the server log had no exception. That shows the app starts and is served. It does not show what a browser draws, and it does not cover any click. The ingest panel and the motion panel have their own checks: a small drive goes through each in 0.2 s (click and rerun). Ingesting a full-density drive through the panel was not measured; the command-line ingest took 44.7 s and 43.6 s on 25 Oct and 91.5 s in a clean-clone run on 27 Oct.

Deliverables

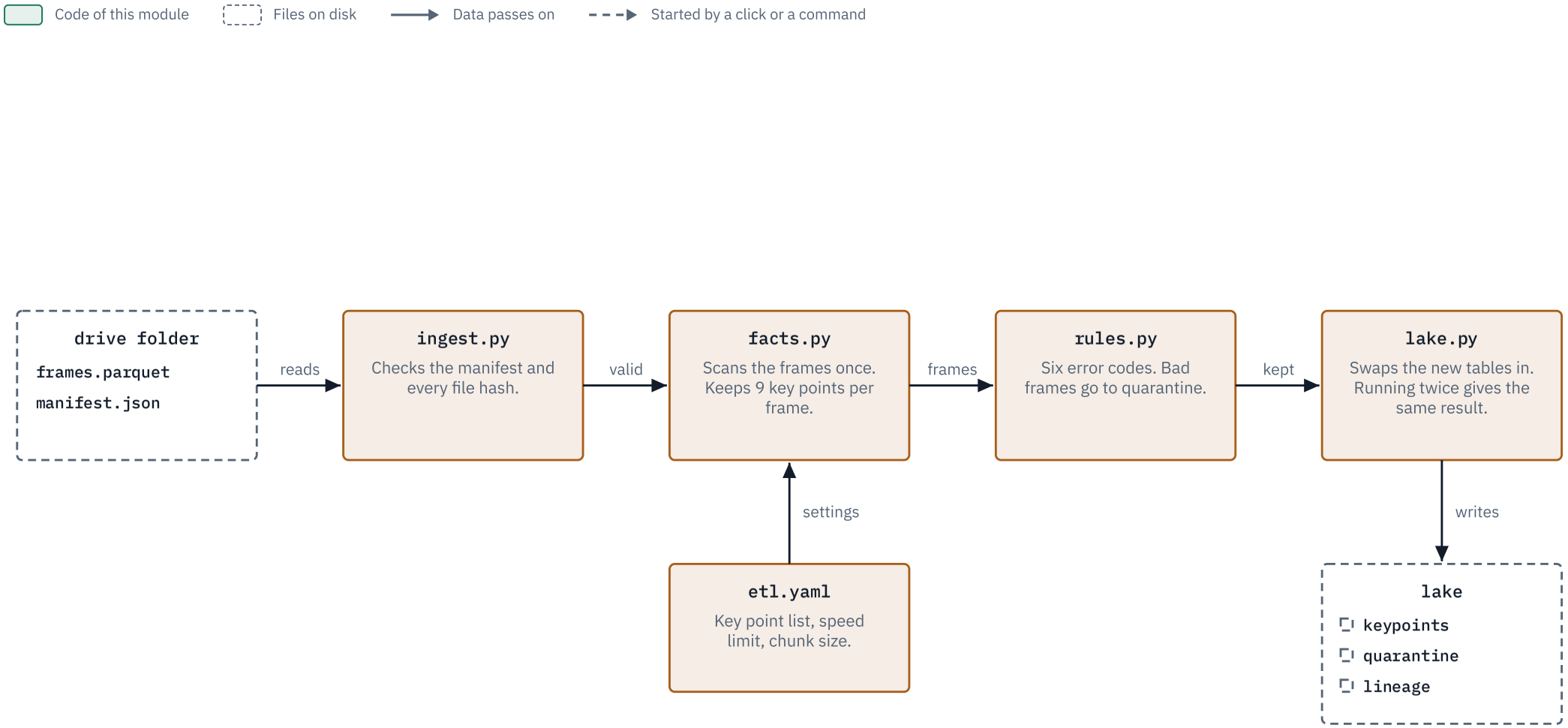
This report is an illustrated overview. The documents the assignment asks for are short files in the repository's `docs/` folder.

WHAT THE ASSIGNMENT ASKS FOR	WHERE IT IS
At least three use cases	`docs/USE_CASES.md`: six, one implemented
A data source (mock)	Module 1 · Generator. The demo drive has 1,000 frames of 1,000,000 points
ETL for the chosen use case	Module 2 · ETL + Loader and Module 3 · Use case
A data warehouse with a generic structure	The lake: keypoints, quarantine, lineage and motion tables (Module 2 tab)
BI for the user	Module 4 · BI: the Streamlit app
Why each technology, and the alternatives	`docs/ARCHITECTURE.md`: the technology choices table
Supporting documentation: error codes	`docs/ERROR_CODES.md`
Supporting documentation: architecture and design	`docs/ARCHITECTURE.md`
Supporting documentation: release notes (description, usage, known issues, coming soon)	`docs/RELEASE_NOTES.md`
LLM usage: model, main prompts, approach	`docs/LLM_USAGE.md`

Module 1 structure

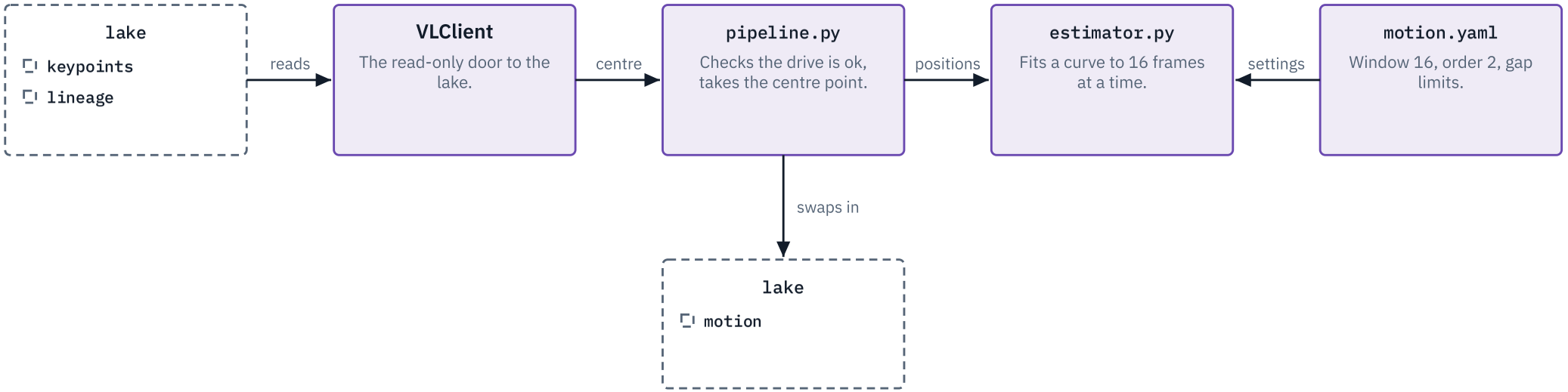


Module 2 structure



Module 3 structure

Code of this module · Files on disk · Data passes on · Started by a click or a command



Module 4 structure

